

Circle all of the statements below that are true:

- a. $\log(n) = O(n)$ True
- b. $n = O(\log n)$
- c. $\log^2(n) = O(n)$ True
- d. $n = O(\log^2 n)$
- e. $\log(n) = \Omega(n)$
- f. $n = \Omega(\log n)$ True
- g. $5n^3 + 7n + 13 = O(n^5)$ True
- h. $5n^3 + 7n + 13 = \Omega(n^3)$ True
- i. $5n^3 + 7n + 13 = O(n)$
- j. $\log(n!) = \Theta(n \log(n))$ True
- k. $n^{1/2} = O(\log n)$
- l. $2^n = \Omega(n!)$

Answer the following:

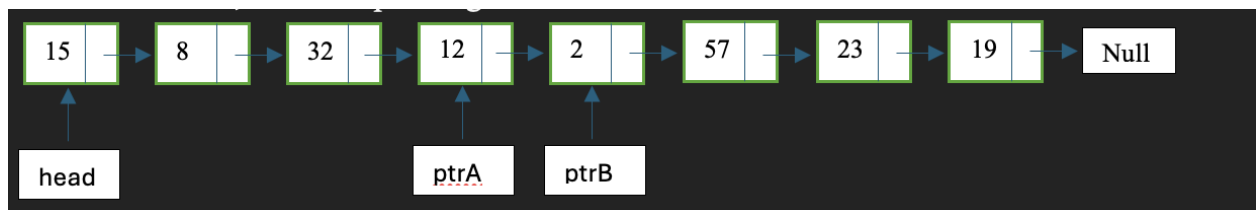
- m. big-Oh runtime for Merge Sort = $O(n \log n)$
- n. big-Oh runtime for Selection Sort = $O(n^2)$
- o. big-Oh runtime for Quick Sort = $O(n^2)$
- p. lower bound on comparison-based sorting problem = $\Omega(n \log n)$
- q. big-Oh runtime for binary search in a sorted array = $O(\log n)$

Linked List

For these questions, the data type 'node' refers to the type defined by the class

```
class node
{
    public:
    int data;
    node * next;
};
```

Consider the linked list below with node pointers ptrA and ptrB pointing to the specified nodes in the list, and head pointing to the first node in the list.



- a. Assume ptrB points to the node with 2 in the data field and ptrA points to the node previous to that node. Write the C++ statements that delete the node containing 2 and update ptrB so that it points to the node that followed 2 after deletion. (Your solution must not create a memory leak)

```
ptrA->next = ptrB->next; // bypass the node with 2
node* temp = ptrB;       // save node to delete
ptrB = ptrB->next;       // update ptrB to next node
delete temp;             // free memory
```

- b. Assume ptrB points to the node with 2 in the data field and ptrA points to the node previous to that node.

Write the C++ statements that:

Delete the node containing 2

Insert a new node containing 63 in the same position where 2 was removed

(Your solution must not create a memory leak.)

```
node* newNode = new node;
newNode->data = 63;
newNode->next = ptrB->next;
```

```
ptrA->next = newNode;  
delete ptrB;
```

Sorting

For this question, suppose you have a `linkedList` class that includes methods 'split' and 'merge', already implemented (exactly as was done in the homework):

```
class linkedList{
```

```
Private:
```

```
    class node{
```

```
    Public:
```

```
        double data;
```

```
        node * next;
```

```
        node * prev;
```

```
    };
```

```
    node * head; //A pointer to the first node in the list
```

```
    node * tail; //A pointer to the last node in the list
```

```
Public:
```

```
    //Splits the contents of list A evenly into the two given (initially empty) lists.
```

```
    //If the number of items in the list is odd, puts the extra item in the first list.
```

```
    //Runs in O(n) time, where n is the total number of items in the list.
```

```
    void split(linkedList &A, linkedList &B);
```

```
    //Merge takes 2 sorted lists and merges them
```

```
    //into your (initially empty) list to create one sorted list.
```

```
    //Runs in O(n) time, where n is the total number of items in the two given lists.
```

```
    void merge(linkedList &A, linkedList &B);
```

```
};
```

a. Given this linked list class and the methods `split` and `merge`, add a method `sort()` to the `linkedList` class that sorts the list into ascending order using a divide-and-conquer strategy.

(If the list contains fewer than 2 elements, your method should return immediately. You may use the methods `split` and `merge` to help achieve this.)

b. Give a recurrence equation describing the runtime of your sorting algorithm.

c. Provide the Big-Oh runtime of your sorting algorithm.

```

a. void linkedList::sort(){
    // Base case: empty list or single element
    if (head == nullptr || head->next == nullptr)
        return;

    // Create two empty lists
    linkedList A, B;

    // Split current list into A and B
    split(A, B);

    // Recursively sort the sublists
    A.sort();
    B.sort();

    // Merge the sorted sublists back into this list
    merge(A, B);
}

```

b. $T(n) = 2T(n/2) + cn$

c. $T(n) = O(n \log n)$

Binary Search Tree

For the following question, assume binary trees consist of nodes from the following class:

```
class node {  
    public:  
        double data;  
        node * left;  
        node* right;  
        node (double x) {data = x; left = nullptr; right = nullptr; }  
};
```

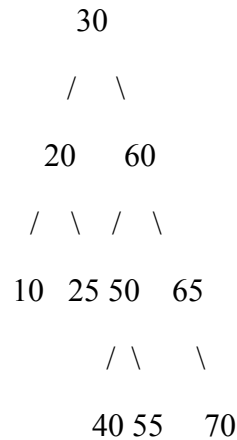
Write a C++ function 'node* buildTree(vector<double> &A, int start, int end)' that takes as input a sorted vector A, and creates a binary search tree using the elements in A from indices start to end, inclusive. The function should return a pointer to the root of the newly created tree, and the tree's height must be $O(\log n)$, where n is the number of items in the tree.

```
node* buildTree(vector<double> &A, int start, int end){  
    // Base case: no elements  
    if (start > end)  
        return nullptr;  
    // Choose middle element  
    int mid = start + (end - start) / 2;  
    // Create root node  
    node* root = new node(A[mid]);  
    // Recursively build left and right subtrees  
    root->left = buildTree(A, start, mid - 1);  
    root->right = buildTree(A, mid + 1, end);  
    return root;  
}
```

AVL Tree

a. Given the following sequence of values inserted in the order shown into an initially empty AVL Tree:

40, 20, 60, 10, 30, 25, 50, 70, 65, 55



b. Suppose the following AVL Tree node class is given:

```
class AVLNode {
```

```
Public:
```

```
    int data;
```

```
    AVLNode* left;
```

```
    AVLNode* right;
```

```
    int height;
```

```
};
```

Implement the following:

(You may assume helper functions such as getHeight() are allowed.)

1. Function to compute and return the balance factor of a node (check difference between left and right subtree)
2. Right rotation function and return root of subtree
3. Left rotation function and return root of subtree
4. BONUS: Insert() function
(inserts a value into the AVL Tree and restores balance using rotations when necessary)

```

1. int getBalance(AVLNode* node){
    if (node == nullptr)
        return 0;
    return getHeight(node->left) - getHeight(node->right);
}

2. AVLNode* rightRotate(AVLNode* p){
    AVLNode* A = p;
    AVLNode* B = A->left;
    AVLNode* BR = B->right;

    p = B;
    B->right = A;
    A->left = BR;

    A->height = max(getHeight(A->left), getHeight(A->right)) + 1;
    B->height = max(getHeight(B->left), getHeight(B->right)) + 1;

    return p;
}

3. AVLNode* leftRotate(AVLNode* p){
    AVLNode* A = p;
    AVLNode* B = A->right;
    AVLNode* BL = B->left;

    p = B;
    B->left = A;
    A->right = BL;

    A->height = max(getHeight(A->left), getHeight(A->right)) + 1;
    B->height = max(getHeight(B->left), getHeight(B->right)) + 1;

    return p;
}

```



```

4.AVLNode* insert(AVLNode* node, int key){
    // Standard BST insertion
    if (node == nullptr)
        return new AVLNode{key, nullptr, nullptr, 1};
    if (key < node->data)
        node->left = insert(node->left, key);
    else if (key > node->data)
        node->right = insert(node->right, key);
    else
        return node; // duplicates not allowed

    // Update height
    node->height = 1 + max(getHeight(node->left),getHeight(node->right));
    int balance = getBalance(node);
    // Left Left Case
    if (balance > 1 && key < node->left->data)
        return rightRotate(node);
    // Right Right Case
    if (balance < -1 && key > node->right->data)
        return leftRotate(node);
    // Left Right Case
    if (balance > 1 && key > node->left->data){
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }

    // Right Left Case
    if (balance < -1 && key < node->right->data){
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }
    return node;
}

```